

Sistema de Arquitetura Moderna para Simulações LVC

Uma arquitetura event-driven Open Source com Event Sourcing e CQRS para simulações Live–Virtual–Constructive em Smart Cities.

AUTOR

Rafael Rodrigues Pereira

ORIENTADOR

Prof. Dr. Fabio Kawaoka Takase

REPOSITÓRIO
& DOCUMENTAÇÃO



Roteiro da apresentação

- | | |
|----|------------------------------------|
| 01 | O gargalo da interoperabilidade |
| 02 | Fundamentos: Event Sourcing e CQRS |
| 03 | Arquitetura ponta a ponta |
| 04 | Cenários LVC em operação |
| 05 | Validação, limites e contribuições |

TESE, EM UMA FRASE

É possível alcançar interoperabilidade LVC com tecnologias [Open Source](#), sem sacrificar desempenho, rastreabilidade ou reprodutibilidade.

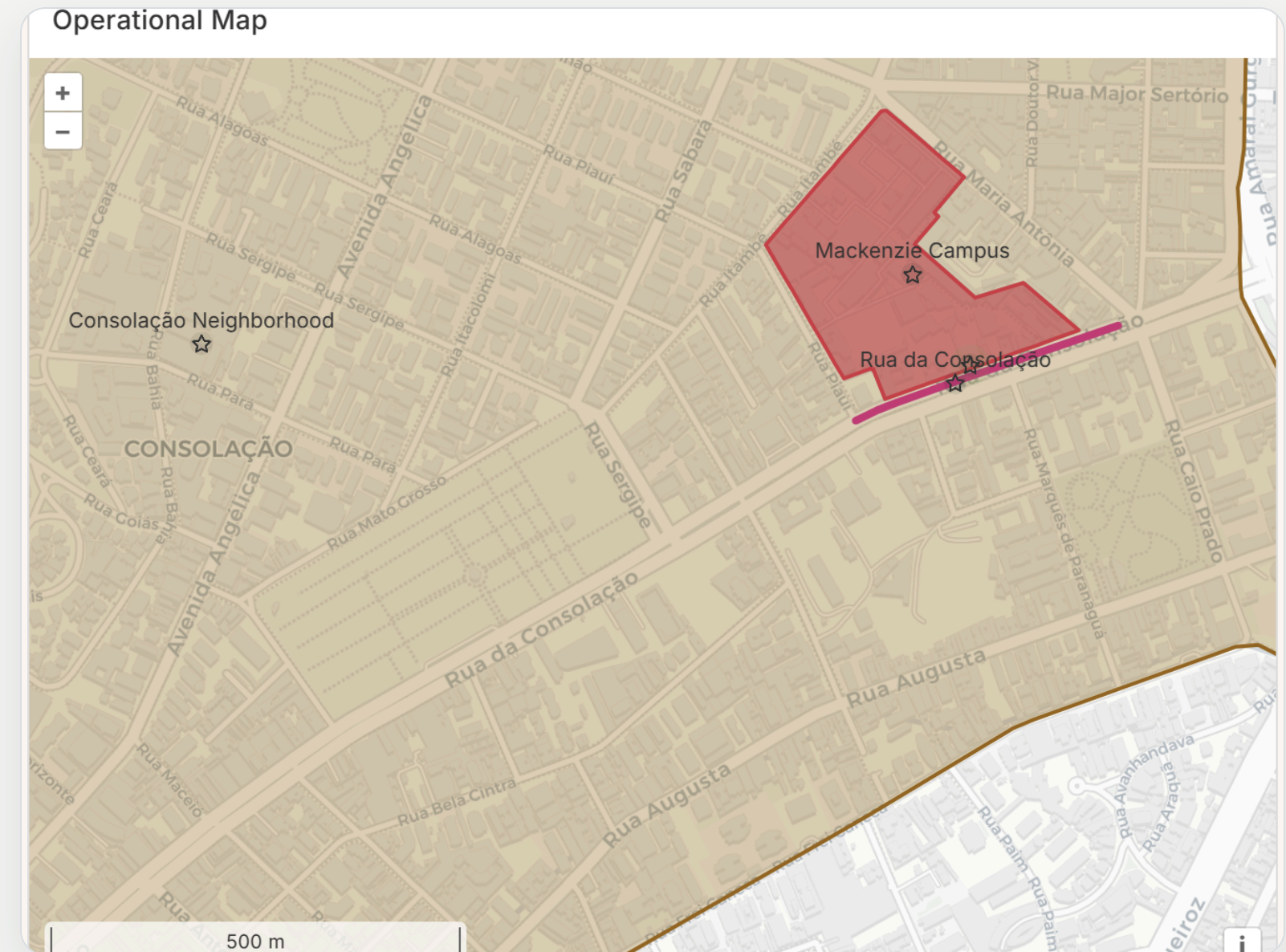
PRÓXIMA SEÇÃO

O gargalo da interoperabilidade

Smart Cities precisam de LVC. HLA e DIS travam a pesquisa acadêmica.

Gêmeos digitais urbanos exigem integrar dados reais, modelos virtuais e simulações construtivas, o paradigma **Live–Virtual–Constructive**.

- Padrões consagrados (HLA, DIS) vêm do domínio militar.
- Dependem de **RTI proprietária** e licenciamento caro.
- Curva de aprendizado alta antes de produzir ciência.
- Integram mal com o **ecossistema aberto** e customizado.



Área operacional • Campus Higienópolis • Mackenzie

Um objetivo geral, quatro específicos.

Geral. Propor e validar uma arquitetura de software **event-driven** Open Source para simulações LVC sobre o Campus Living Lab da Mackenzie.

01

Revisar a literatura

LVC, Event Sourcing, CQRS, gêmeos digitais urbanos.

02

Definir a arquitetura

Event-driven, CQRS, contrato canônico em .proto.

03

Implementar a plataforma

Seis módulos independentes e políglotas em Open Source.

04

Validar com cenários LVC

Quatro cenários integrados sobre o campus Higienópolis.

02

PRÓXIMA SEÇÃO

Fundamentos: Event Sourcing e CQRS

Live, Virtual, Constructive: três modos de simular.

LIVE

Pessoas reais
Sistemas reais

Pessoa	real
Sistema	real

Ex.: ônibus passando no ponto Caio Prado
· navio no canal · alagamento detectado pela CGESP.

VIRTUAL

Pessoas reais
Sistemas simulados

Pessoa	real
Sistema	simulado

Ex.: operador navegando pelo Gêmeo Digital em Vaadin sobre a planta do campus.

CONSTRUCTIVE

Pessoas simuladas
Sistemas simulados

Pessoa	simulada
Sistema	simulado

Ex.: drone autônomo cumprindo missão dentro do AirSim, planejamento por NIM/Gemma.

• Event Sourcing

• CQRS

• gRPC + Protocol Buffers

HLA / DIS vs. arquitetura proposta.

CRITÉRIO	HLA / DIS	PROPOSTA
Licença	RTI proprietária · custo de adoção	Sem licenciamento de infraestrutura
Curva de aprendizado	OMT, FOM e dezenas de classes antes do 1º experimento	gRPC + Protocol Buffers, padrões de mercado
Integração com OSS moderno	Ecossistema fechado em si	Rust · Java · Python · Docker · Grafana
Multi-linguagem	Bindings limitados (OMT/FOM)	Stubs Java + Python + Rust por proto-shared
Observabilidade nativa	Não há, exige tooling adicional	Read path direto para Grafana

Por que **event-driven**. Por que **Open Source**.

EVENT-DRIVEN

- Sistemas LVC são **reativos por natureza**: navios, ônibus, alertas, missões.
- Eventos discretos modelam o mundo real com fidelidade temporal.
- O log de eventos é, de graça, um **audit trail** imutável.
- Replay é uma operação de primeira classe.

OPEN SOURCE

- **Sem RTI proprietária**, zero custo de licenciamento na infraestrutura.
- Stack moderno: Rust, Java 25, Python, Docker.
- LLM **plugável**: NVIDIA NIM (cloud) ou Ollama Gemma local, quem replica escolhe.
- Comunidade ativa, ferramental maduro, reproduzível no meio acadêmico.

O estado é o log: eventos imutáveis, em sequência.

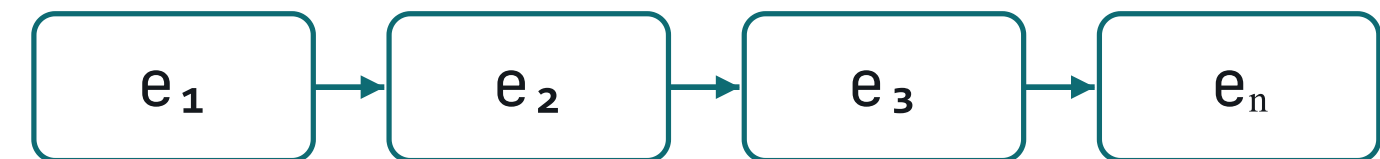
Em vez de guardar apenas o estado atual, o sistema persiste cada mudança como um **evento imutável**. O presente é a soma do log — e qualquer instante passado se reconstrói por replay.

POR QUÊ

- **Rastreabilidade** — cada decisão do sistema é um evento auditável.
- **Reprodutibilidade** — replay integral reconstrói qualquer cenário.

VS. TRADICIONAL HLA mantém estado volátil e distribuído entre federados — sem log central, sem replay.

log imutável • append-only →



replay

Estado = Σ eventos

Separar quem escreve de quem lê.

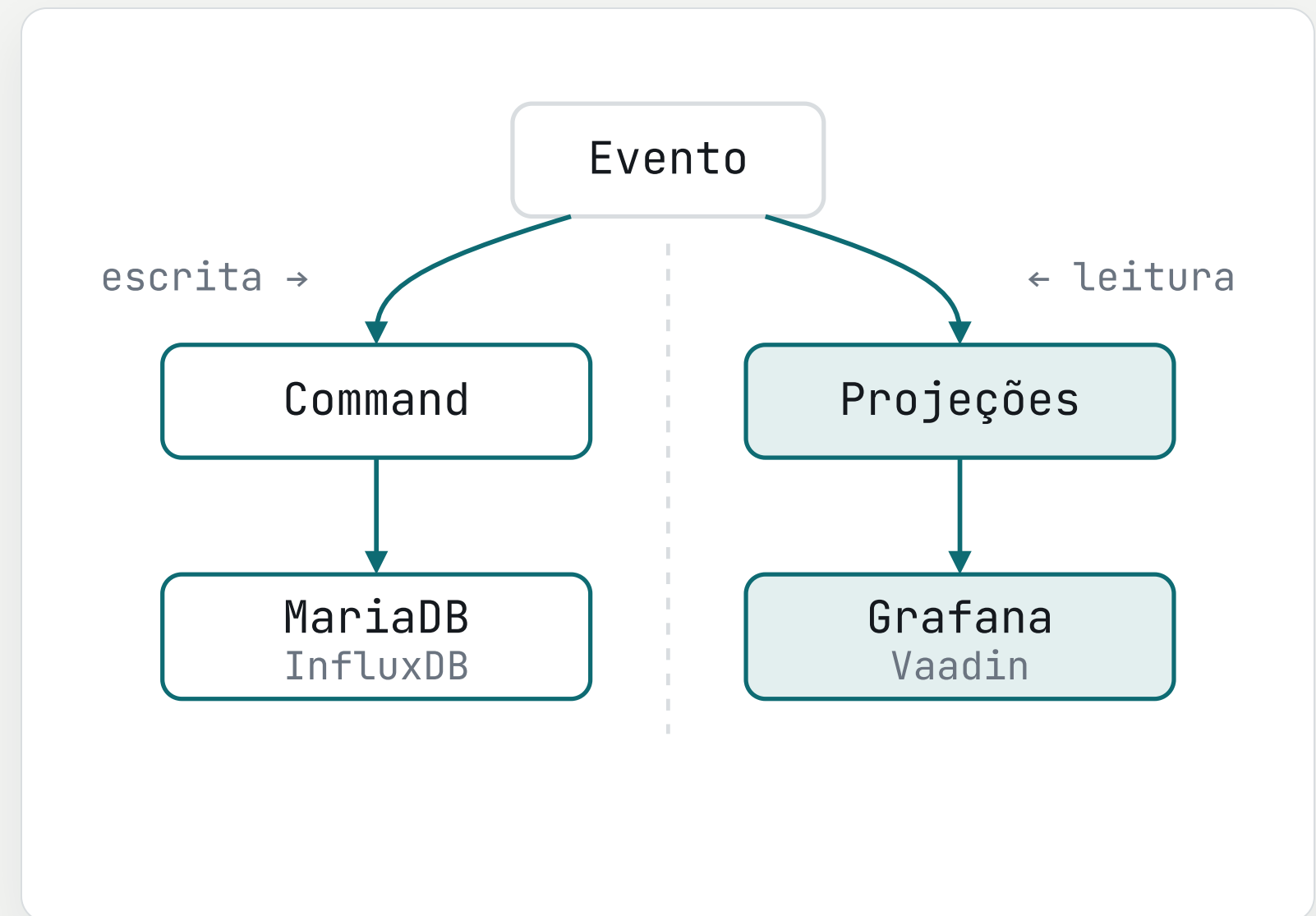
Command Query Responsibility

Segregation divide a aplicação em dois caminhos. A escrita ingere eventos de alta frequência com durabilidade; a leitura serve **projeções materializadas** com baixa latência.

POR QUÊ

- **Write path** — ingestão Live/Constructive + Transactional Outbox.
- **Read path** — dashboards e Gêmeo Digital sobre projeções.

VS. TRADICIONAL Um modelo único de leitura/escrita não separa as cargas — e ambas competem pelo mesmo recurso.



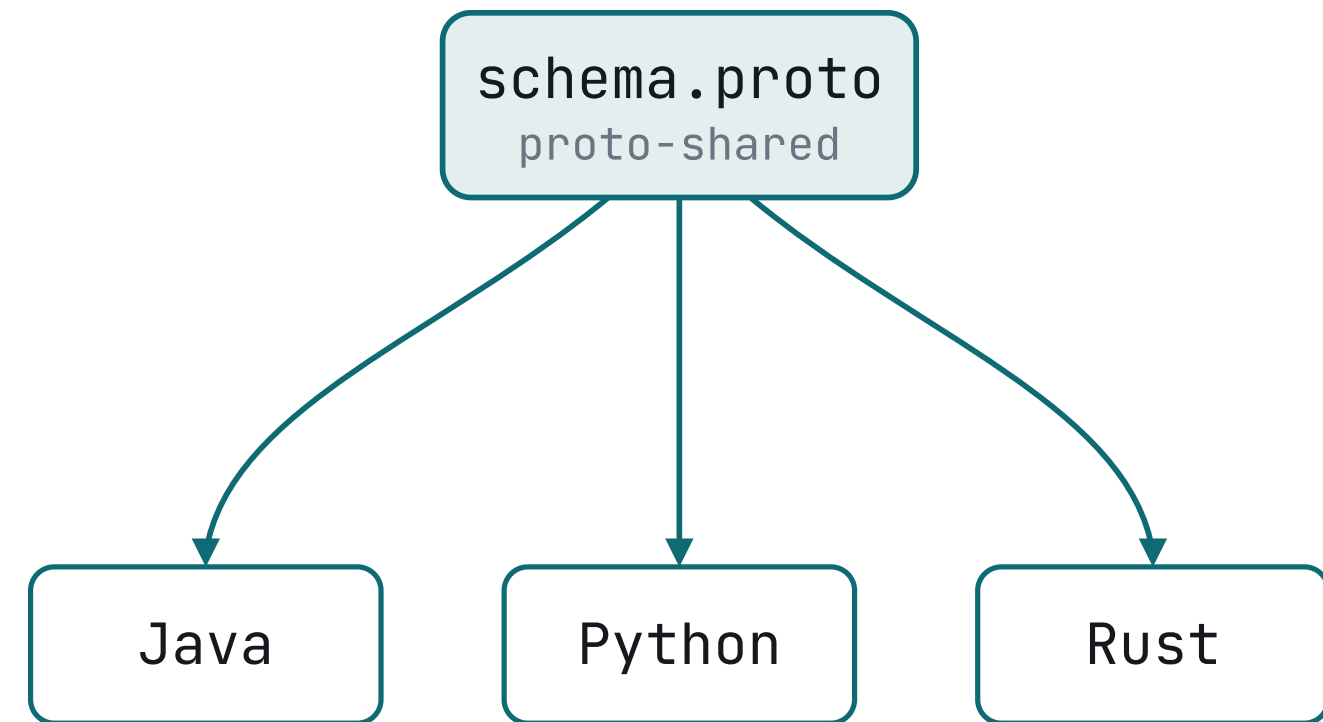
Um .proto. Três linguagens. Zero ambiguidade.

Protocol Buffers definem o contrato (IDL); o gRPC transporta sobre HTTP/2 com streaming bidirecional. Um único módulo proto-shared é o ponto de verdade contratual.

POR QUÊ

- 1 arquivo .proto → stubs **Java**, **Python** e **Rust**.
- Build automatizado — Maven + protoc, Docker prost/tonic.

VS. TRADICIONAL HLA exige bindings específicos de C++/Java — cada linguagem, um esforço à parte.



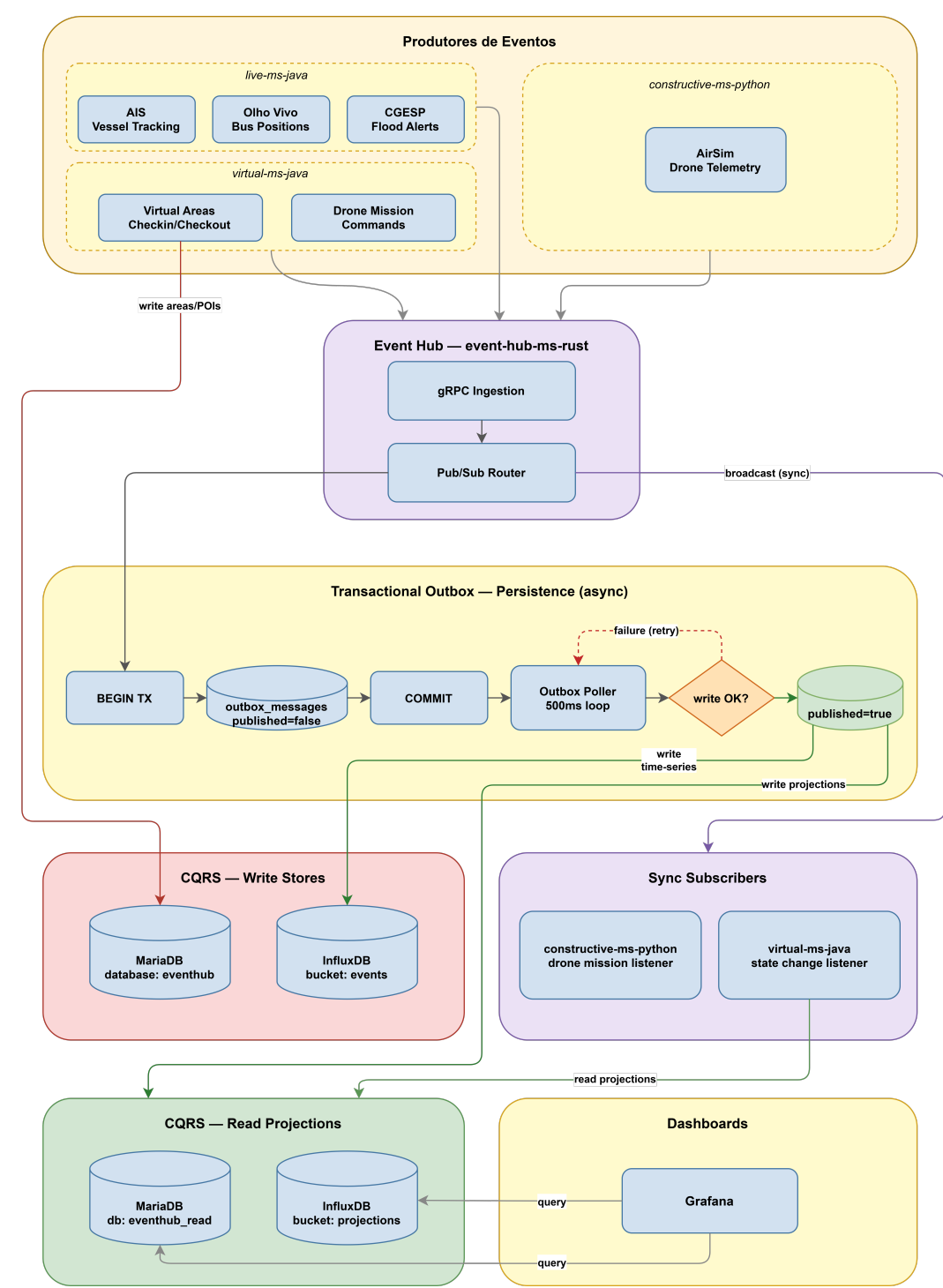
1 contrato → 3 stubs gerados

03

PRÓXIMA SEÇÃO

Arquitetura ponta a ponta

A plataforma, de ponta a ponta, em três camadas.



01 CAMADA • INGESTÃO

Produtores de eventos

Adaptadores Live, Virtual e Constructive publicam eventos via gRPC, em três linguagens, num contrato único.

`live-ms-java • virtual-ms-java • constructive-ms-python`

02 CAMADA • PERSISTÊNCIA

Event Hub + Transactional Outbox

O hub roteia num Pub/Sub uniforme; o Outbox garante at-least-once, persistindo e despachando sob uma única transação.

`event-hub-ms-rust • MariaDB • InfluxDB • poller 500ms`

03 CAMADA • CONSUMO

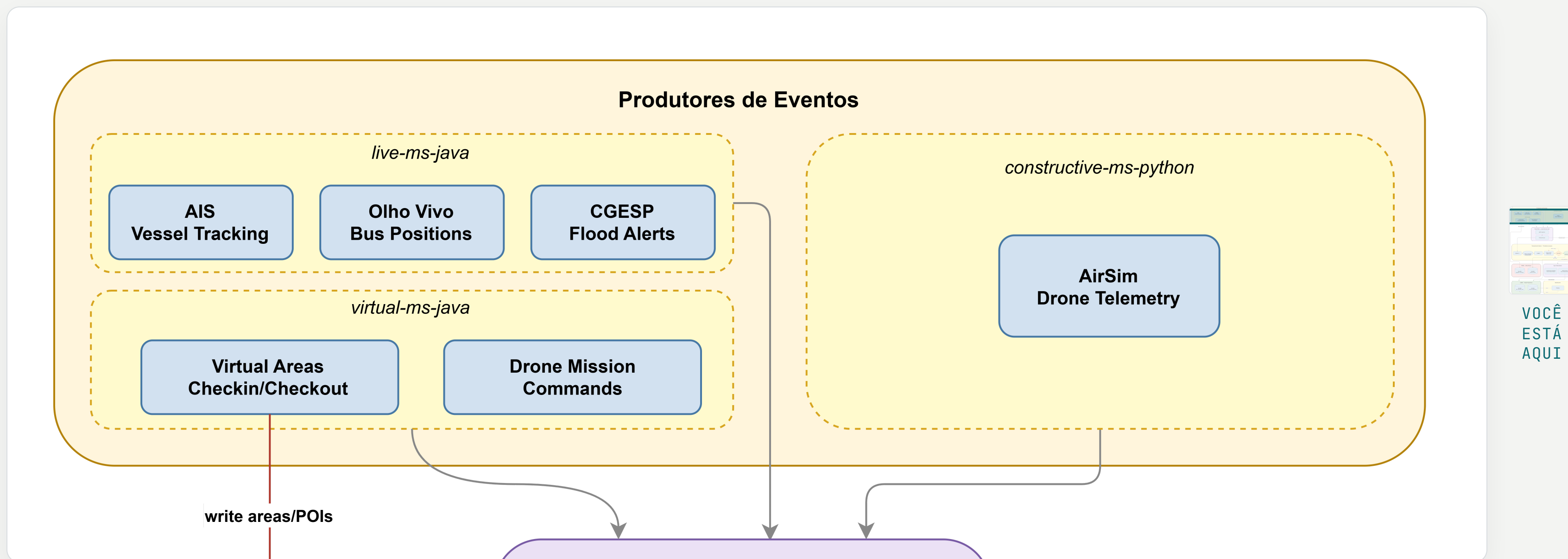
CQRS + Sync Subscribers + Grafana

Write Stores e Read Projections separam escrita de leitura; subscribers fecham ciclos de missão e o Grafana consulta as projeções.

`write stores • eventhub_read • projections • Grafana`

Stack reativo • `Virtual Threads` (JEP 444) • Circuit Breaker (Resilience4j) • back-pressure

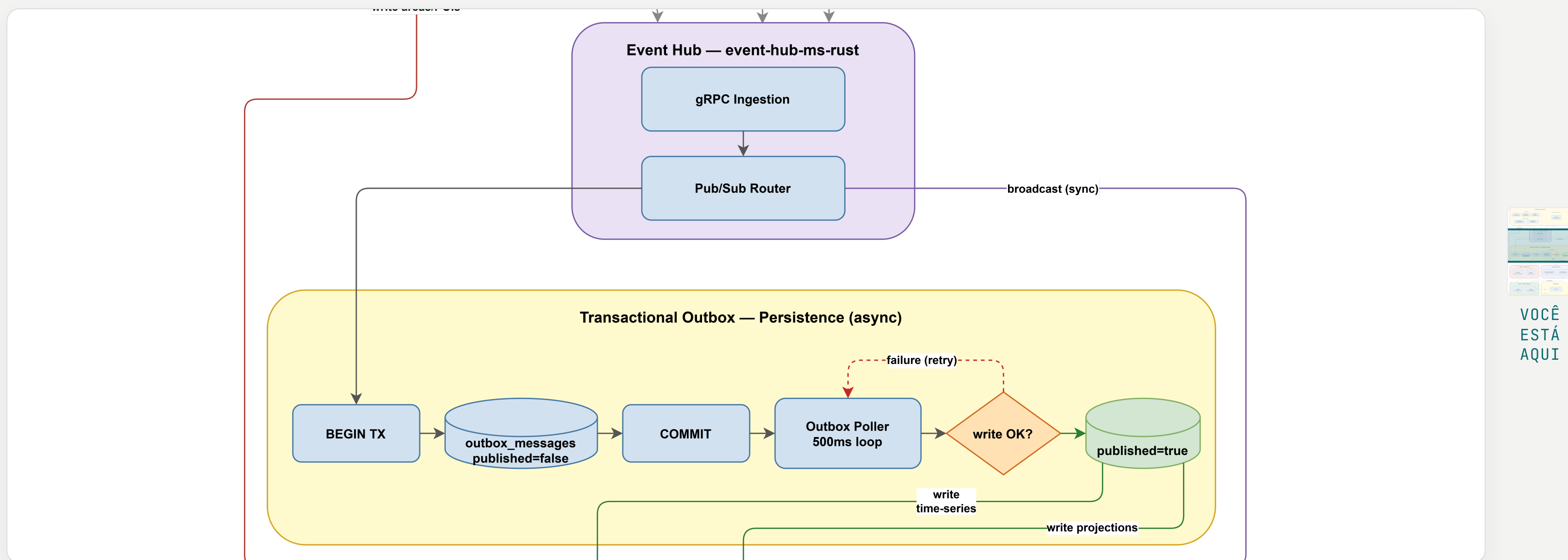
Ingestão: três famílias de produtores, um único barramento.



Cinco produtores em três linguagens publicam no `event-hub-ms-rust` via gRPC.

Adapter → gRPC → Pub/Sub → broadcast

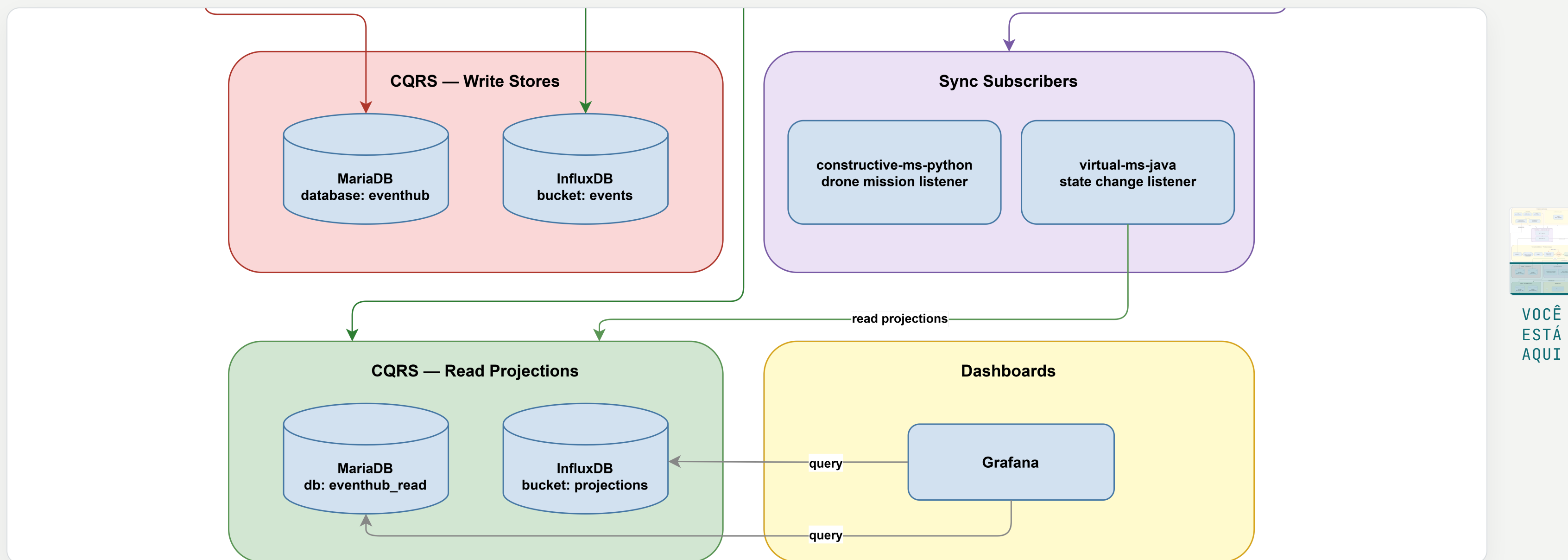
Persistência: o Transactional Outbox garante at-least-once.



Persistir e publicar são **uma única transação**; o Poller repete a cada 500 ms até o sucesso.



Consumo: projeções de leitura, subscribers síncronos e dashboards.



Sync Subscribers fecham ciclos de missão; Read Projections materializam visões para o Grafana.

published → write proj → **Read Proj** → Grafana

Seis módulos independentes, abordagem poliglota.

MÓDULO	STACK	RESPONSABILIDADE
proto-shared	.proto · gRPC · stubs Java/Python/Rust	Contrato canônico, ponto único de verdade
event-hub-ms-rust	Rust · Transactional Outbox	Barramento de eventos · caminho quente
live-ms-java	Java 25 · Spring Boot 4	Adaptadores AIS · OlhoVivo · CGESP
virtual-ms-java	Java · Vaadin	Gêmeo Digital · SPA do campus Higienópolis
constructive-ms-python	Python · AirSim · NIM · Ollama Gemma	Missões Constructive · drone autônomo
infra-local-shared	Docker Compose	Orquestração local reprodutível

Um caminho de escrita, um caminho de leitura.

WRITE PATH

Ingestão de alta frequência

Produtores Live + Constructive

AIS · OlhoVivo · CGESP · AirSim



event-hub-ms-rust

Caminho quente + Transactional Outbox



Event Store

append-only · log imutável

Otimizado para throughput, ordenação e atomicidade transacional.

READ PATH

Projeções materializadas

Projeções · eventhub_read

Read models otimizados por consulta



virtual-ms-java

Vaadin · Gêmeo Digital



Grafana

Dashboards de observabilidade

Otimizado para forma e latência de consulta, evolui independentemente.



04

PRÓXIMA SEÇÃO

Cenários LVC em operação

Cadastro de áreas e POIs no Gêmeo Digital.

A interface virtual-ms-java define a **verdade espacial** dos cenários: cada área, polygon, circle, corredor, vira gatilho CHECKIN_CHECKOUT ou MISSION_TRIGGER.

Virtual Areas

Areas

New Area

Name	Type	Monitors	Action	Status	Actions
Mackenzie Campus	POLYGON	DRONE	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Rua da Consolação	CORRIDOR	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Consolação Neighborhood	POLYGON	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Caio Prado C/B Stop Area	CIRCLE	BUS	MISSION_TRIGGER	Active	Edit Deactivate Delete
Santos Port	CIRCLE	VESSEL	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete

Ingestão Live e eventos de check-in / check-out.

ADAPTADOR 01

AIS

Embarcações no canal, REST autenticada, cadência minuto-a-minuto.

•

ais

ADAPTADOR 02

SPTrans OlhoVivo

Frota de ônibus, posição em tempo real, API pública com cadastro.

•

olhovivo

ADAPTADOR 03

CGESP

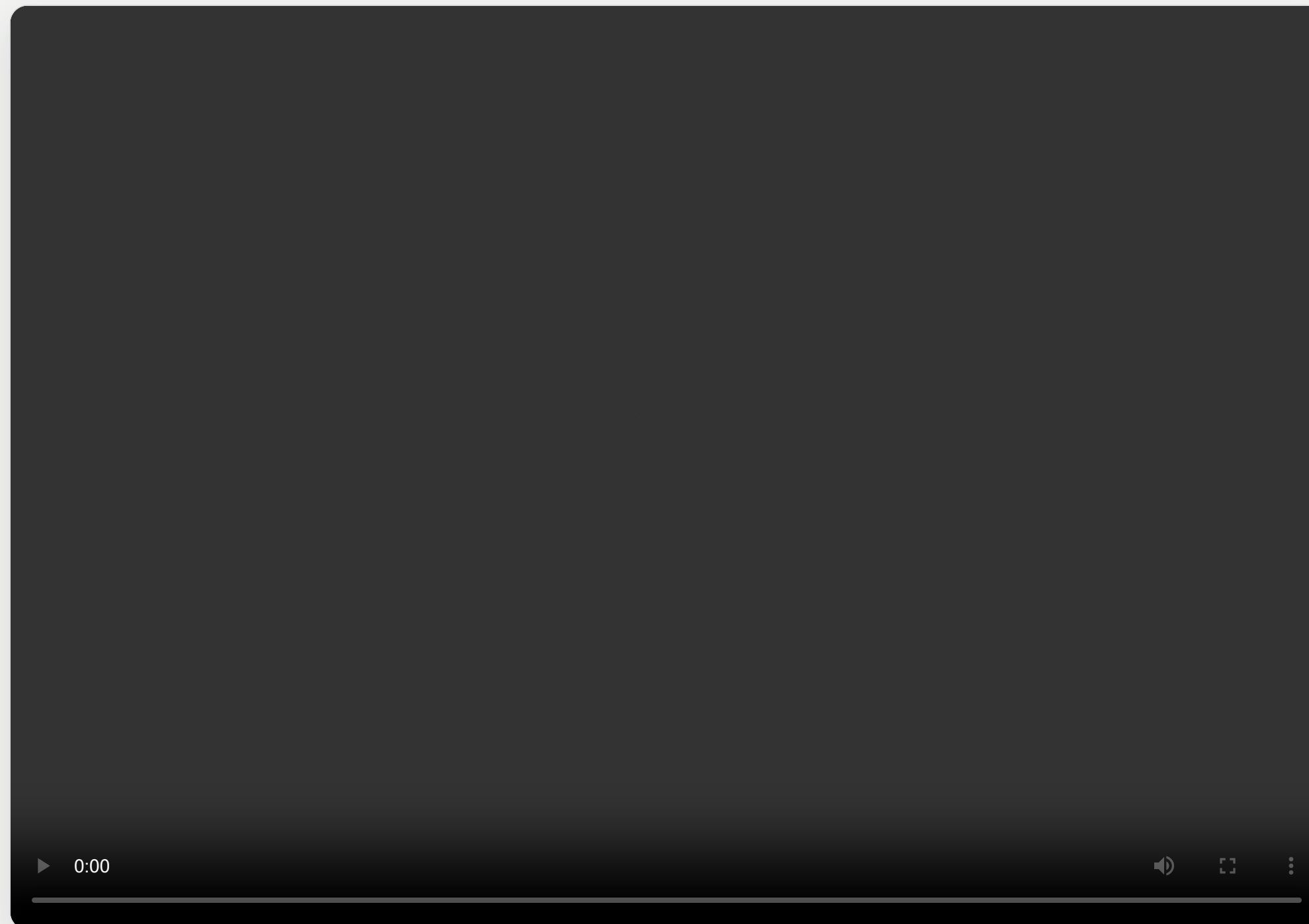
Alagamentos de São Paulo, web scraping resiliente.

•

cgesp

Event Log									
event_id	area	entity_id	entity_type	event_type	lvc	source	lat	lon	occurred_at ↓
8d7d4efb-d9b1-4000-8000-000000000000	Santos Port	710009040	vessel	CHECKIN	LIVE	virtual-ms-	-24.1	-46.4	2026-05-20 22:56:13
a2001371-37e1-4000-8000-000000000000	Santos Port	405000377	vessel	CHECKIN	LIVE	virtual-ms-	-24.2	-46.4	2026-05-20 22:56:13
90eb67a2-1271-4000-8000-000000000000	Rua da Consolação	81512	bus	CHECKOUT	LIVE	virtual-ms-	-23.5	-46.7	2026-05-14 01:41:45
04e44fef-9811-4000-8000-000000000000	Rua da Consolação	81512	bus	CHECKIN	LIVE	virtual-ms-	-23.5	-46.6	2026-05-14 01:40:42
95ec75e3-f931-4000-8000-000000000000	Consolação Neighborhood	81512	bus	CHECKIN	LIVE	virtual-ms-	-23.5	-46.6	2026-05-14 01:40:42
25c75d5b-9b11-4000-8000-000000000000	Consolação Neighborhood	81506	bus	CHECKOUT	LIVE	virtual-ms-	-23.5	-46.6	2026-05-14 01:39:39
ad28798c-13b1-4000-8000-000000000000	Caio Prado C/B Stop Area	81506	bus	CHECKOUT	LIVE	virtual-ms-	-23.5	-46.6	2026-05-14 01:39:39
79a6f1ed-6791-4000-8000-000000000000	Rua da Consolação	81506	bus	CHECKOUT	LIVE	virtual-ms-	-23.5	-46.6	2026-05-14 01:39:39
35a5d20c-0511-4000-8000-000000000000	Rua da Consolação	81506	bus	CHECKIN	LIVE	virtual-ms-	-23.5	-46.7	2026-05-14 01:38:36
cd913bd2-8ec1-4000-8000-000000000000	Caio Prado C/B Stop Area	81506	bus	CHECKIN	LIVE	virtual-ms-	-23.5	-46.7	2026-05-14 01:38:36
4ec09fad-3041-4000-8000-000000000000	Consolação Neighborhood	81506	bus	CHECKIN	LIVE	virtual-ms-	-23.6	-46.7	2026-05-14 01:36:29
5ffd0746-de01-4000-8000-000000000000	Santos Port	flood-a1b2c3d4-0005-0000-0000-000000000000	flood_area	MOVE	VIRTUAL	virtual-ms-	-24.0	-46.3	2026-05-14 01:33:40
6ce16d33-2fff-4000-8000-000000000000	Santos Port	MEGA-SANTOS	vessel	CHECKOUT	VIRTUAL	virtual-ms-	24.0	36.3	2026-05-14 01:32:09
ca064399-2911-4000-8000-000000000000	Santos Port	MEGA-SANTOS	vessel	CHECKIN	VIRTUAL	virtual-ms-	-24.0	-46.3	2026-05-14 01:31:56

Missão do drone disparada por evento Live.



ENCADEAMENTO

- 01 Ônibus chega ao ponto Caio Prado (OlhoVivo).
- 02 `virtual-ms-java` detecta entrada na área e publica evento de gatilho.
- 03 `constructive-ms-python` planeja trajetória com NIM / Ollama Gemma.
- 04 Drone voa no AirSim e captura fotos do alvo.
- 05 Cada passo é republicado no Event Hub, auditoria fim-a-fim.

Pipeline Constructive: do evento ao voo.

01	Evento de gatilho · Live → Virtual Ônibus chega ao ponto Caio Prado · <code>virtual-ms-java</code> publica o evento de entrada.	fonte real (OlhoVivo) → área cadastrada no Cenário 1
02	constructive-ms-python monta o contexto Posição do drone, localização do alvo, política da missão e restrições espaciais do campus.	contexto é estruturado em JSON, não texto livre
03	Modelo de linguagem → sequência de waypoints NVIDIA NIM (cloud) ou Ollama Gemma (local), escolha do operador.	saída exigida: JSON estrito · esquema versionado
04	Validação + execução no AirSim Schema check rejeita alucinações; waypoints válidos viram chamadas da API do AirSim.	trajetória inválida → missão abortada com evento de erro
05	Telemetria republicada no Event Hub Cada waypoint atingido + fotos capturadas + eventuais colisões.	auditoria fim-a-fim · base do dashboard Grafana

Grafana em tempo real: a evidência empírica.



PAINEL SUPERIOR • VIDA DO SISTEMA

entidades por área • taxa de check-in / check-out • mapa histórico • log de auditoria

PAINEL INFERIOR • ALERTAS

alagamentos CGESP • colisões do drone no AirSim • fotos capturadas pelas missões

05

PRÓXIMA SEÇÃO

Validação, limites e contribuições

Quatro dimensões, quatro evidências.

01

Rastreabilidade

Estado reconstruível a partir de um log imutável; replay como operação de primeira classe.

evidência · Event Sourcing

02

Reprodutibilidade

Ambiente completo sobe com docker compose up em qualquer máquina compatível.

evidência · infra-local-shared

03

Interoperabilidade multi-linguagem

Um único .proto gera stubs Java, Python e Rust, três linguagens, um só contrato.

evidência · proto-shared

04

Integração com fontes reais

Três adaptadores Live em modalidades distintas: REST autenticada, web scraping e API com cadastro.

evidência · AIS · OlhoVivo · CGESP

Os quatro objetivos específicos, fechados ✓



Revisar a literatura

LVC, Event Sourcing, CQRS e gêmeos digitais urbanos, capítulo de fundamentação.



Definir a arquitetura

Arquitetura event-driven com CQRS, Event Sourcing e contrato em .proto.



Implementar a plataforma

Seis módulos políglotas rodando em conjunto via Docker Compose.



Validar com cenários LVC

Quatro cenários demonstrados sobre dois dashboards Grafana do campus Higienópolis.

Quatro contribuições, e o caráter Open Source como uma a mais.

01

Arquitetura event-driven Open Source para LVC

Alternativa concreta e replicável a HLA / DIS no contexto acadêmico.

02

O módulo proto-shared como ponto único de verdade

Um padrão de contrato canônico para plataformas políglotas, replicável fora deste trabalho.

03

Integração de três fontes Live reais e públicas

AIS, SPTrans OlhoVivo e CGESP, três modalidades distintas, em produção contínua.

04

Quatro cenários LVC ponta-a-ponta

Exercitados sobre o campus Higienópolis, com observabilidade Grafana auditável.

+ Repositório público com Deepwiki, extensível por qualquer pesquisador.

Honestidade científica: o que ficou em aberto.

LIMITAÇÕES

- Validação **estatística comparativa** entre sinal Live e modelos Virtual/Constructive não foi conduzida.
- Avaliação de **escalabilidade horizontal** sob carga massiva não foi feita, testes em máquina única.

TRABALHOS FUTUROS

- Estudo estatístico comparativo Live vs. Virtual/Constructive (fidelidade do gêmeo).
- Avaliação de escalabilidade horizontal sob carga e perfis de latência.
- Expansão do catálogo de adaptadores Live.
- Outros modelos de planejamento para o Constructive.

31 ENCERRAMENTO

Obrigado. Perguntas?

REPOSITÓRIO

github.com/tcc-modern-lvc-arch

DOCUMENTAÇÃO

deepwiki.com/tcc-modern-lvc-arch

AUTOR • ORIENTADOR

Rafael Rodrigues Pereira • Prof. Dr. Fabio Kawaoka Takase

